

FAQ Section

Wie kann die Architektur beschrieben werden?

- Womit interagiert der Endnutzer
 - Der Endnutzer arbeitet über einen Browser mit dem in Next.js entwickelten Frontend. Von dort werden REST-Aufrufe an das Express-Backend geschickt. Direkten Kontakt zur Datenbank gibt es nicht.
- Womit interagiert der Endnutzer nicht
 - Der Endnutzer hat keinerlei direkte Verbindung zu Server oder Datenbank. Alle technischen Details wie Session-Verwaltung oder Datenbankzugriffe laufen vollständig im Backend ab, ohne dass der Nutzer darauf zugreifen kann.
- Wie sind Applikation und Server verbunden
 - Das Frontend läuft in einem Next.js-Server, der als Client im Browser ausgeführt wird, und kommuniziert per HTTP/REST mit dem Express-Server auf Port 4000. In der `NEXT_PUBLIC_API_URL` wird definiert, unter welcher Adresse die API erreichbar ist.
- Wie sind Server und Datenbank verbunden bzw tauschen Daten aus
 - Der Express-Server nutzt das `pg`-Modul, um über einen Connection-Pool auf die PostgreSQL-Datenbank zuzugreifen. Die Verbindungskonfiguration liest er aus `credentials.json`. Anfragen werden durch einen `opossum`-Circuit-Breaker geleitet, der bei Fehlern den Pool neu aufbaut.
- wie könnte ich eine Grafik mit folgenden Komponenten bauen: Server, Datenbank, Applikation, Sessions, Browser, Endnutzer
 - Ein mögliches Architekturdiagramm zeigt den Browser des Nutzers, der über HTTPS auf das Next.js-Frontend zugreift. Dieses ruft per REST den Express-Server auf, der Sessions in `express-session` verwaltet und über `pg` mit der PostgreSQL-Datenbank spricht. Pfeile zwischen den Komponenten verdeutlichen die Datenflüsse.

Wie funktionieren Sessions?

- Wo werden wie Sessions verwaltet (bspw. beim Checkout oder Login)?

- Die Sessions werden serverseitig mit `express-session` gehalten. Beim Login speichert der Server im Session-Objekt die Nutzer-ID und Berechtigungen. Beim Checkout legt er ein Unterobjekt `checkout` an, in dem Versand- und Zahlungsdaten während des Kaufvorgangs liegen.
- Wie sind die Sessions aufgebaut?
 - Jede Session besteht aus einem Cookie mit dem Namen `sid`, das im Browser hinterlegt ist, und einem dazugehörigen Servereintrag im MemoryStore. Dort liegen Attribute wie `userId`, `email`, diverse Rollenrechte sowie das `checkout`-Objekt. Die Daten sind also in einem JS-Objekt strukturiert.
- Welche Daten werden in welcher Session wie lange gespeichert?
 - Die Login-Session speichert Nutzerdaten eine Stunde lang, da die Cookie-Option `maxAge` auf 3600000 Millisekunden steht. Daten des Checkout-Vorgangs werden zusätzlich im `checkout`-Objekt abgelegt und nach 15 Minuten Inaktivität verworfen.
- Wo werden die Daten genau gespeichert die in einer Session sind, macht das der Browser oder wer sonst?
 - Die eigentlichen Sessiondaten liegen ausschließlich im Node.js-Server und werden standardmäßig in dessen Speicher abgelegt. Der Browser hält nur die Session-ID im Cookie. Bei jedem Request sendet er dieses Cookie mit, sodass der Server die zugehörigen Daten finden kann.

Wie genau werden Bilder gespeichert?

- Wie werden die Bilder von .png bzw .jpeg in ein Format umgewandelt was die Datenbank speichert?
 - Der Server nimmt hochgeladene PNG- oder JPEG-Dateien über `multer` entgegen. Da wir den Speicher auf `memoryStorage` gesetzt haben, liegen sie bereits als Buffer im Arbeitsspeicher. Dieser Binärpuffer wird anschließend direkt in die Spalte `image_data` der Tabelle `images` geschrieben.
- in welcher Form bekommt der Server die Bilddaten und wie werden diese dann in images gespeichert?
 - Durch das Multer-Middleware bekommt `server.js` die hochgeladenen Dateien in `req.file` beziehungsweise `req.files` als Buffer zusammen mit dem MIME-Type. Jeder Upload erhält eine

UUID als ID. Anschließend wird `INSERT INTO images (id, image_data, image_type)` ausgeführt, sodass Buffer und Typ persistiert werden.

- wie werden Bilder aus der Datenbank geholt
 - Zum Abruf stellt das Backend die Route `/image/:id` bereit. Dort wird per `SELECT image_data, image_type FROM images WHERE id = $1` das gespeicherte Binary geladen. Danach setzt der Server den Content-Type auf den gespeicherten MIME-Wert und sendet die Daten als HTTP-Response zurück.
- wie erfolgt die Umwandlung vom Format der Datenbank in ein Bild?
 - Im Browser geschieht keine Konvertierung: das Backend liefert das Bild bereits mit korrektem MIME-Type aus. Der Browser interpretiert den Content-Type und stellt das Binary direkt als Bild dar. Es wird also nur übertragen und nicht nochmals transformiert.

Wie genau funktionieren Hooks?

- Was ist der Sinn von Hooks?
 - React-Hooks ermöglichen es, in Funktionskomponenten zustandsbehaftete Logik und Nebeneffekte zu kapseln. Mit `useState` werden lokale Zustände verwaltet, `useEffect` führt Code bei bestimmten Änderungen aus. Dadurch bleibt der Code schlanker und wiederverwendbar.
- Wie genau werden Hooks in diesem Code genutzt?
 - In den React-Seiten wie `login.jsx` und den Checkout-Komponenten werden `useState` und `useEffect` eingesetzt, um Formularfelder zu steuern und API-Daten zu laden. Beispielsweise speichert `useState` die E-Mail im Login-Formular und `useEffect` ruft Versand- oder Zahlungsoptionen ab.

Wie funktioniert der Auto-Restart und Circuit Breaker durch Opossum und P2M?

- Was ist der Sinn von den zwei Integrationen?
 - PM2 überwacht den Node-Prozess und startet ihn automatisch neu, falls er abstürzt. Opossum kapselt einzelne Funktionen – hier vor allem

Datenbankzugriffe – in einen Circuit Breaker und verhindert, dass wiederholte Fehler das gesamte System lahmlegen.

- Was macht ein Circuit Breaker von Opossum?
 - Der Circuit Breaker überwacht Fehlerraten und Antwortzeiten der gekapselten Funktion. Überschreiten die Fehler ein vordefiniertes Limit, wird der Breaker geöffnet. In dieser Phase werden Anfragen sofort abgelehnt, bis nach Ablauf von `resetTimeout` ein neuer Versuch gestartet wird.
- Wie genau weiß der Circuit Breaker, dass das backend/frontend grade nicht funktioniert und startet die apps durch?
 - Opossum misst Erfolgs- und Fehlerschwellen beim Ausführen der eingekapselten Funktion, hier etwa eines DB-Queries. Wird die Funktion mehrmals nacheinander mit Fehlern beantwortet oder überschreitet die Antwortzeit das Timeout, geht der Breaker in den offenen Zustand. Dadurch werden weitere Aufrufe blockiert, bis ein Retry ansteht. Ein Neustart der App erfolgt über PM2.
- Wie genau kann p2m oder opossum die apps durchstarten?
 - PM2 führt den Node-Prozess als Daemon aus und überwacht Exit-Codes sowie Speicherverbrauch. Stürzt der Prozess ab oder beendet er sich selbst, startet PM2 automatisch einen neuen Worker. Opossum selbst startet die App nicht neu; es sorgt aber dafür, dass fehlerhafte Funktionen nicht mehr ausgeführt werden, bis sie wieder stabil sind.
- Was sind die zeitintervalle, in denen P2m und Opossum die App überprüfen, ob sie läuft
 - Im Projekt werden keine speziellen PM2-Intervalle definiert, da PM2 permanent den Prozessstatus beobachtet. Der Circuit Breaker führt jedoch alle zehn Sekunden einen Datenbank-Healthcheck durch und öffnet sich fünf Sekunden lang nach jedem Fehler (`resetTimeout`), bevor ein erneuter Verbindungsversuch erfolgt.
- Wie kann die App dennoch abstürzen?
 - Trotz Circuit Breaker und PM2 können unbehandelte Exceptions, Speicherlecks oder Logikfehler im Code zum Absturz führen. Wenn beispielsweise ein API-Endpunkt eine falsche Query ausführt oder Endlosschleifen erzeugt, beendet sich der Prozess. PM2 startet ihn zwar neu, aber der Fehler muss im Quellcode behoben werden.

Wie funktioniert Rendering mit Nest.js/Express und allen genutzten Technologien?

- Welche Rendering-Strategie wird auf dieser Webseite genutzt
- Die Anwendung verwendet Next.js. Seiten wie `checkout/shopping-cart.jsx` nutzen `getServerSideProps`, um Inhalte serverseitig vorzubereiten und anschließend im Browser zu hydratisieren. Dadurch erhalten wir eine serverseitig gerenderte Seite, die direkt mit Daten geladen wird.
- Wie wird sichergestellt auf der Websiete, dass Rendern nicht zu lange braucht
- Next.js rendert die meisten Seiten bereits auf dem Server und lädt im Frontend nur die notwendigen Daten nach. Beim Seitenwechsel zeigt `_app.jsx` ein `LoadingOverlay`, damit der Nutzer Feedback bekommt, während asynchrone `fetch`-Aufrufe laufen.
- Welche Vorteile beim Rendering würden andere Frameworks bieten, die nicht genutzt wurden?
- Frameworks wie Svelte oder Angular besitzen zum Teil eine kleinere Laufzeit bzw. bringen eine stärkere Typisierung mit. Für dieses Projekt wurde jedoch React/Next.js gewählt, weil es gut in die bestehende Express-Struktur passt und SSR sowie Routing out of the box mitbringt.

Was sind die konkreten Unterschiede von Nest.js zu...

- Svelte
- Nest.js ist ein Backend-Framework. Svelte arbeitet rein im Browser und kompiliert Komponenten zu minimalem JavaScript.
- SvelteKit
 - SvelteKit erweitert Svelte um Routing und Serverfunktionen. Im Gegensatz zu Nest.js liegt der Fokus auf dem Frontend.
- Vue
 - Vue ist ebenfalls ein Frontend-Framework mit virtueller DOM und eignet sich für reaktive Benutzeroberflächen. Nest.js hingegen strukturiert Node.js-Server.
- Angular

- Angular bietet ein umfassendes Ökosystem mit TypeScript und Dependency-Injection für Frontend-Apps. Nest.js nutzt ähnliche Konzepte auf der Serverseite.
- React
 - React bildet die Grundlage unseres Next.js-Frontends. Im Gegensatz zu Nest.js kümmert sich React ausschließlich um die UI und nicht um Serverlogik.

Integrationen von Backend, Frontend und Datenbank

- Welche Libraries und Module wurden benutzt, um eine Kommunikation zwischen Frontend, Backend und Datenbank zu ermöglichen
- Im Frontend wird `fetch` genutzt. Die Basis-URL ist in `config.js` definiert. Das Backend setzt auf Express mit `pg` für PostgreSQL und `express-session` zur Verwaltung der Sitzungen.
- Wie genau wird diese Kommunikation technisch ermöglicht und wie genau funktionieren die Frameworks technisch?
- Der Browser sendet HTTP-Requests an das Express-Backend. Dort greifen die Routen auf die Datenbank zu und setzen Session-Informationen. React nutzt Hooks wie `useAuth` und `useCart`, um diese Daten zu laden und im Zustand der Anwendung zu speichern.
- welche konkreten Methoden machen dies im Code? generiere die Code Snippets als Beispiel dafür aus einem der Files in Frontend, Backend (server/server.js)
- Beispiel Frontend (`useCart.js`):

```
const res = await fetch(`${API_BASE_URL}/cart-items`,  
  { credentials: 'include' });
```

- Beispiel Backend (`server.js`):

```
app.post('/login-user', async (req, res) => {  
    const result = await client.query(`SELECT ...  
FROM users WHERE email = $1`, [email.trim()]);  
});
```

Weitere Fragen

1. Wie wird beim Laden der App der Loginstatus ermittelt?

- In `useAuth.js` wird zuerst ein eventuell gespeicherter Nutzer aus `localStorage` gelesen und anschließend `/session-status` auf dem Backend abgefragt, um die Session zu validieren.

2. Welche Aufgabe hat `getServerSideProps` in den Checkout-Seiten?

- Diese Funktion prüft vor dem Rendern, ob eine gültige Session existiert. Fehlt sie, wird der Nutzer sofort auf die Login-Seite umgeleitet.

3. Wie aggregiert `useCart.js` die Ticketmengen?

- Nach dem Laden der Warenkorbdaten werden reguläre und Begleittickets pro Veranstaltung zusammengezählt und in `counts` abgelegt.

4. Was passiert, wenn die Reservierungszeit im Checkout abläuft?

- `shopping-cart.jsx` startet einen Countdown von 15 Minuten. Läuft dieser ab, löscht das Frontend den Checkout über einen `DELETE`-Aufruf und zeigt ein Ablauf-Modal an.

5. Wo ist die Backend-URL im Frontend hinterlegt?

- `config.js` exportiert `API_BASE_URL`, das auf `process.env.NEXT_PUBLIC_API_URL` oder `http://localhost:4000` fällt.

6. Wie funktioniert die Suchleiste in der Navigationsleiste?

- `nav-bar.jsx` überwacht das Eingabefeld per `useEffect` und ruft nach

kurzer Verzögerung die Route `/search-tours` auf, um Vorschläge anzuzeigen.

7. Wie wird die Server-Session geprüft?

- Im Backend existiert die Route `/session-status`, welche die Daten aus der Session liest und dem Frontend als JSON zurückgibt.

8. Wodurch erhält der Benutzer ein Feedback beim Navigieren?

- `_app.jsx` zeigt während eines Routenwechsels ein `LoadingOverlay`, indem es auf `Router.events` reagiert und einen Ladezustand setzt.

9. Wie lädt die Startseite Tour- und Künstlerdaten?

- `index.jsx` ruft in zwei `useEffect`-Hooks die Routen `/tours-with-images` und `/artists-with-images` ab und speichert die Ergebnisse in State-Variablen.

10. Wie werden Merkzeichen im Antrag bearbeitet?

- `DisabilityRequestModal.jsx` lädt beim Wechsel in den Edit-Modus die verfügbaren Merkzeichen über `/disability-marks` und fügt sie via `toggleMark` dem Formularzustand hinzu.